

DAY 5

1. Command Identification – Stage 2

After **Tokenization**, Bash identifies what type of command structure it is dealing with.

```
Stage 1 → Tokenization
Stage 2 → Command Identification
Stage 3 → Expansion
```

2. Simple Commands

Definition

A **simple command** is a basic command consisting of a **command name** followed by its arguments, options, or inputs.

Example

```
echo "Hello"
```

Here:

- `echo` → command name
- `"Hello"` → argument

Another example:

```
ls -l /home
```

- `ls` → command
- `-l` → option/argument
- `/home` → argument

How are simple commands terminated?

Simple commands are typically terminated by **control operators**, such as:

```
newline  
;  
|
```

Example:

```
echo Hello; echo World
```

There are two simple commands separated by `;`.

3. Compound Commands

Definition

A **compound command** is a more complex command structure made up of commands and control logic.

They are commonly used in **shell scripting**.

Compound commands often use **reserved words** to mark their beginning and end.

Example: `if`

```
if [ "$x" -gt 10 ]  
then  
    echo "Greater"  
fi
```

Here:

```
if    → starts the structure  
fi    → ends the structure
```

Why are compound commands important?

They allow scripts to implement:

- Conditional logic
- Loops
- Grouping
- More complex execution flow

Common examples

```
if ... fi
for ... done
while ... done
until ... done
case ... esac
```

Simple command = basic command

Compound command = structured/logic-based command

4. Shell Expansion – Stage 3

After Bash identifies the commands, it performs **expansion** on words before execution.

Expansion transforms the command into the form that Bash will ultimately use.

There are **4 major expansion stages** that occur in a specific order.

5. Four Stages of Expansion

```
1. Brace Expansion
   ↓
2. Parameter / Arithmetic / Command Substitution / Tilde Expansion
   ↓
3. Word Splitting
   ↓
4. Globbing
```

Stage 1: Brace Expansion

Brace expansion happens **first**.

Example

```
echo {a,b,c}
```

Bash expands this into:

```
a b c
```

Another example:

```
echo file{1,2,3}.txt
```

Produces:

```
file1.txt  
file2.txt  
file3.txt
```

Key point

Brace expansion always occurs before the other expansion stages.

6. Stage 2: Parameter, Arithmetic, Command & Tilde Expansion

This stage contains several types of expansion.

A. Parameter Expansion

Used to substitute the value of a variable.

```
name="Alice"  
echo "$name"
```

Result:

```
Alice
```

`$name` is replaced by the value of `name`.

B. Arithmetic Expansion

Used to perform mathematical calculations.

Syntax:

```
$(( expression ))
```

Example:

```
echo $((5 + 3))
```

Output:

```
8
```

C. Command Substitution

Used to replace a command with its output.

Syntax:

```
$(command)
```

Example:

```
echo "Today is $(date)"
```

The `$(date)` part is replaced by the output of `date`.

Modern Bash scripts generally prefer `$(...)` over old-style backticks.

D. Tilde Expansion

The `~` character can represent the user's home directory.

Example:

```
cd ~
```

It represents the current user's home directory.

7. Stage 3: Word Splitting

After the earlier expansions, Bash may perform **word splitting** on the resulting text.

This is particularly important when using **unquoted variable expansions**.

Example:

```
name="John Smith"  
echo $name
```

The value contains a space, so the unquoted expansion can be split into separate words:

```
John  
Smith
```

Important

Quoting can prevent this:

```
echo "$name"
```

Now the value remains a **single word**:

```
John Smith
```

Remember: Word splitting generally happens to unquoted expansion results.

8. Stage 4: Globbing

Globbing is also called **pathname expansion**.

It uses wildcard characters to match filenames.

Common wildcards:

Symbol	Meaning
*	Matches multiple characters
?	Matches one character
[abc]	Matches one character from the set

Example

Suppose the directory contains:

```
file1.txt
file2.txt
notes.txt
image.jpg
```

Command:

```
echo *.txt
```

May produce:

```
file1.txt file2.txt notes.txt
```

The `*.txt` pattern is expanded to matching filenames.

9. Expansion Priority ★

The **order matters**.

Bash performs expansion in this priority order:

B → P → W → G

B = Brace Expansion

P = Parameter/Arithmetic/Command/Tilde Expansion

W = Word Splitting

G = Globbing

So:

Brace → Parameter/etc. → Word Splitting → Globbing

Earlier stages are completed **before later stages**.

10. Left-to-Right Rule

If multiple expansions occur within the **same stage**, Bash processes them **from left to right**.

Example concept

If a command contains multiple parameter expansions:

```
echo "$a $b $c"
```

Bash processes those expansions from **left → right** within that expansion stage.

Remember

Different stages → Fixed priority

Same stage → Left to right
